

Audio Mosquito Detector on an ESP32

On-device detection — 91.6% accuracy in a 91 KB model

Yechan Noh (ML)

July 2026

1. The result

Audio Mosquito Detector on an ESP32 Chip

Runs entirely on an ESP32 microcontroller — no cloud, no phone

91.6%

detection accuracy
(measured on the chip)

91 KB	23,394	90%	2.4 / s
model size (fp32)	parameters	mosquitoes caught	on-chip throughput

Trained on real mosquito recordings (HumBugDB) + real fly wingbeats (InsectSound1000)

An AI that hears a mosquito with **91.6% accuracy**, running entirely on an **ESP32 microcontroller** — no cloud, no phone. The model is **91 KB** and catches **90%** of mosquitoes. The rest of this deck explains what those numbers mean and how we got there.

2. The data — where it comes from

The data: real recordings

Two questions the model must tell apart: mosquito, or not?



Training and test use SEPARATE recordings — so the accuracy is honest, not memorised

We only had to answer one question: **is there a mosquito, or not?** So the model learns from two kinds of real recordings:

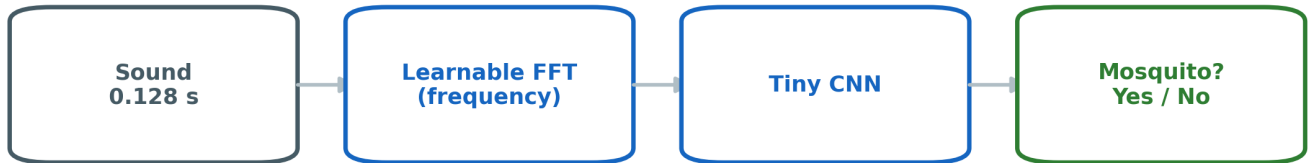
- **Mosquito** — **HumBugDB**, a public dataset (NeurIPS 2021): real mosquitoes recorded with smartphones outdoors, **34 species**.
- **Not mosquito** — two things: (1) **background noise** from the *same* recordings, and (2) **real fly wingbeats** from **InsectSound1000** (public dataset, 2024) — 3 fly species.

Everything is audio. Training and testing use **separate recordings**, so the accuracy is honest — the model can't just memorise a clip.

Why only 3 fly species? Not a shortcut — it's what exists. Insects whose wingbeat overlaps the mosquito band are barely represented in any public dataset; these 3 are essentially all there is. The true look-alikes (midges) have **no public data at all** — which is exactly why recording them ourselves is the project's differentiator.

3. How it works — and it really runs on the chip

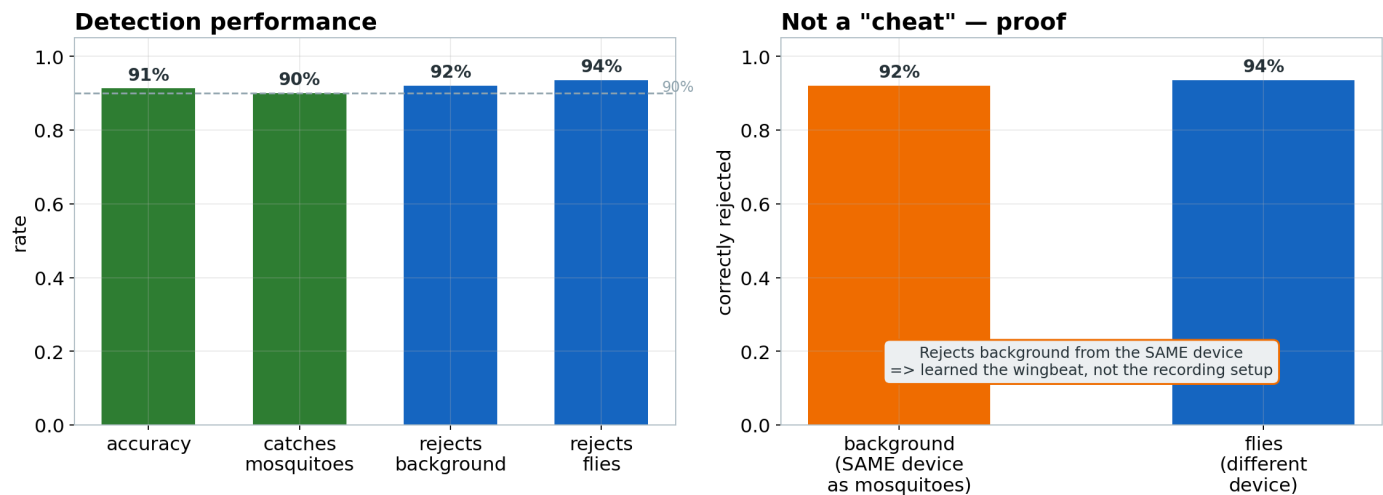
How it works



Runs entirely on the ESP32 chip — no cloud, no phone, no internet

A 0.128-second sound clip → a **frequency step** → a **tiny CNN** → “mosquito: yes / no”. Every step runs on the ESP32 itself. The frequency step is the important one — we come back to it on slide 6.

4. Detection performance — and it is not a shortcut



Catches **90%** of mosquitoes; rejects background at **92%** and flies at **95%** (few false alarms).

Why this is honest: a classifier can cheat by learning “which recording device” instead of “is there a wingbeat.” We deliberately added background recorded on the **same device** as the mosquitoes to the negative class — and the model rejects it 92% of the time. So it learned the actual wingbeat, not the recording setup.

5. What the numbers mean

How the score is calculated

We tested 1,200 sounds on the chip and counted right vs wrong

The four scores

		the model's answer	
		"not mosquito"	"mosquito"
the truth	really MOSQUITO	58 missed	558 correct
	really NOT	541 correct	43 false alarm

Accuracy
everything it got right
91.6%
 $(558 + 541) / 1200$

Catch rate (recall)
of real mosquitoes, how many caught
91%
 $558 / 616$

Few false alarms (precision)
when it says "mosquito", how often right
93%
 $558 / 601$

F1 (balance of the two)
one number so neither can be faked
combines the two above
0.92

We tested **1,200 sounds** on the chip and counted right vs wrong (left). From that table come four scores:

- **Accuracy (91.6%)** — of all 1,200, how many were right: $(558 + 541) / 1200$.
- **Catch rate / recall (91%)** — of the real mosquitoes, how many we caught: $558 / 616$. (We miss ~9%, but surveillance accumulates many readings, so misses are recovered.)
- **Few false alarms / precision (93%)** — when it says "mosquito," how often it's right: $558 / 601$.
- **F1 (0.92)** — a single number combining recall and precision, so you can't win by cheating on just one (e.g. shouting "mosquito!" every time gives 100% recall but terrible precision — F1 catches that).

All four land at **90–93%**, evenly — the model is good at *both* catching mosquitoes and not raising false alarms.

6. The key idea: analyse frequency

Why we analyse frequency (the “FFT” step)

A mosquito is defined by its pitch — so we let the model see pitch directly



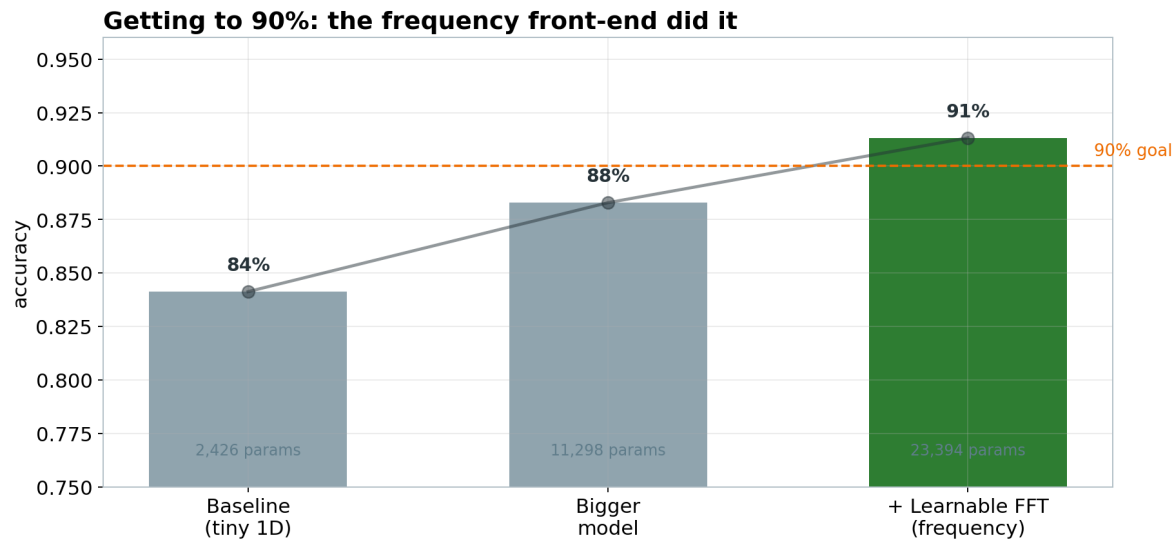
Mosquitoes hum at a specific pitch (~400–800 Hz); flies and background do not.

Our “LearnableFFT” does this split on-chip — and fine-tunes which pitches to watch.

That one idea took the model from 84% to 91% — at no extra size cost.

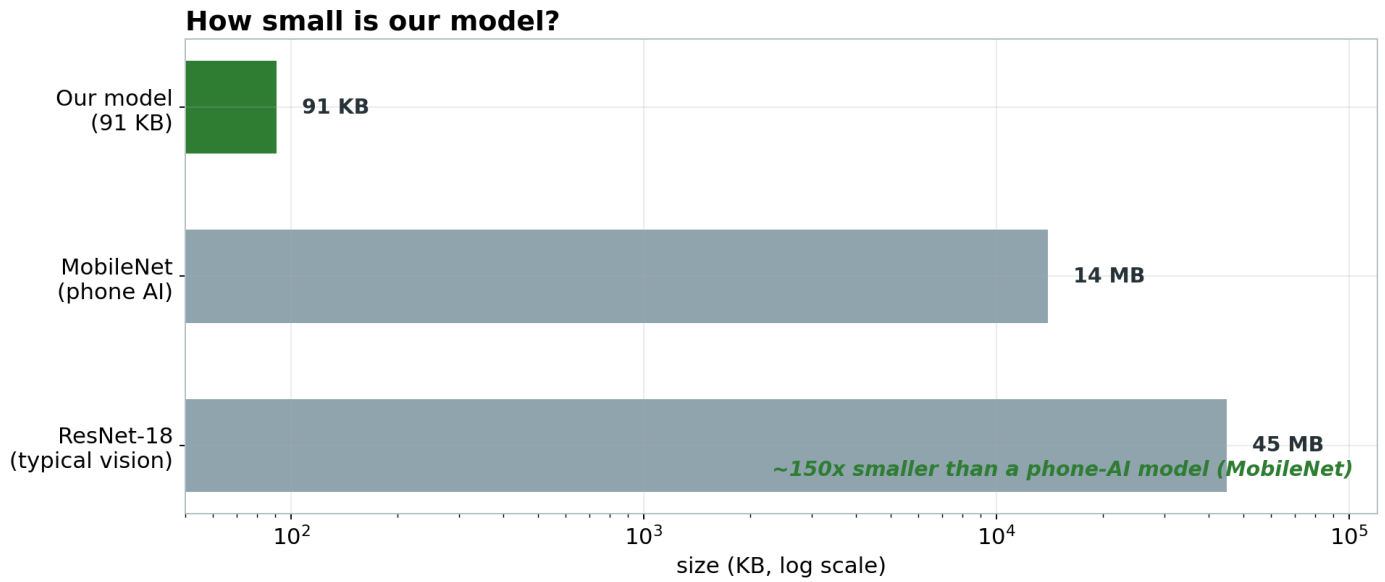
A raw sound wave is just a wiggly line — hard to read. An **FFT** (Fast Fourier Transform) splits it into its **pitches**: how much low hum, mid tone, high whine. A mosquito’s wingbeat sits in a narrow band (~400–800 Hz); flies and background don’t. So the question “is it a mosquito?” becomes the easy “is there a peak in the mosquito band?”

Our **LearnableFFT** does this on the chip and even fine-tunes which pitches to watch.



That frequency step is what carried the model from 84% → 91% — more than just making the model bigger (which stalled at 88%). The right *representation* beat raw size.

7. How small is the model?



At **91 KB** (fp32 weights) the model is **~150× smaller** than a phone-AI model (MobileNetV2, 14 MB) and **~500× smaller** than a typical vision model (ResNet-18, 45 MB). That is why it fits an ESP32 and sips almost no power.

Anticipated questions

Why sound, not a camera? Audio needs only a cheap microphone and almost no power — a good fit for a battery sensor. (An optical version is feasible later; audio simply had the richer public data to build on first.)

It misses ~10% of mosquitoes — is that OK? For surveillance, decisions accumulate over many windows, so the effective detection rate is much higher; a miss on one window is caught on the next.

Can it tell which species? Not in this version — this is detection only (mosquito vs not). Naming the exact species from sound is a much harder, separate problem (different species overlap in wingbeat frequency), out of scope here.

Next steps

- **Record chironomid midges ourselves.** They're the one field look-alike that really overlaps mosquitoes in frequency, and **no public dataset has them** — so recording them gives us data nobody else has, then we fine-tune.
- **Connect a live microphone.** The chip already does the maths; wiring in a mic is a straightforward integration step.